

Title

***Automated CVE Intelligence Alerts for SOC
Teams using Python***

Abstract

This project focuses on automating the daily tracking, extraction, comparison, and reporting of newly published and updated CVEs. Cybersecurity professionals often rely on manual browsing of CVE feeds, which is time-consuming and prone to missed updates. This system automates the entire workflow by fetching CVE records from the API, comparing them with previously stored data, identifying newly published and updated vulnerabilities, extracting critical fields such as CVSS score, severity, CWE, and affected products, and generating a structured email notification. The solution ensures that users receive accurate and timely updates only when necessary—avoiding duplicate or empty emails. The project is designed to be lightweight, modular, and extensible, enabling future integration with AI components such as NLP summarization, RAG-based querying, and automated prioritization.

Introduction

A Common Vulnerabilities and Exposures (CVE) entry identifies publicly disclosed cybersecurity vulnerabilities. Organizations must monitor CVE activity to stay informed about emerging threats, especially those affecting their software stack. However, manually checking CVE feeds is inefficient and does not scale with growing daily volumes. This project solves the problem by creating a fully automated CVE tracking and notification pipeline.

The system automatically identifies:

- New CVEs published today
- CVEs updated today
- Differences between old and new versions
- Critical fields such as CVSS score, severity, CWE, exploit status, and affected vendors

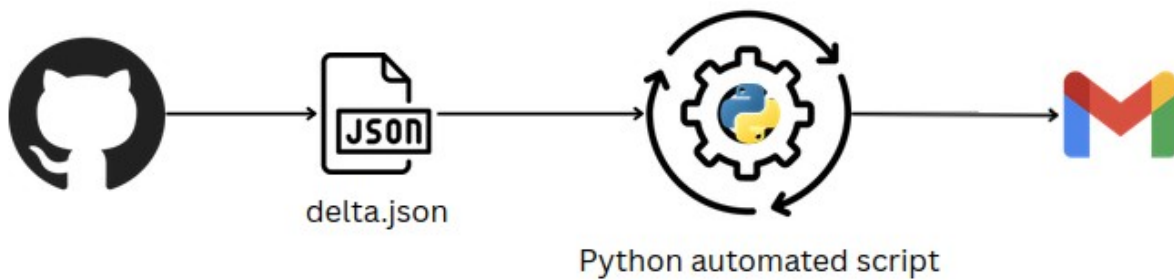
The objective is to build a reliable automation tool used by SOC teams, blue team analysts, penetration testers, and researchers to maintain awareness of emerging vulnerabilities.

Approach: Delta-Based Client Synchronization

My script uses a common method for synchronizing large datasets efficiently, often called **delta synchronization**.

- **Source of Truth:** The primary data source is the **CVEProject/cvelistV5** GitHub repository, specifically accessed via the delta.json file.
 - <https://github.com/CVEProject/cvelistV5/blob/main/cves/delta.json>
 - which gets updated before in the NVD database.
- **Change Detection (Delta):** Instead of downloading the entire CVE list every time (which would be massive and slow), the script downloads the small delta.json file. This file acts as a manifest, listing only the **new** and **updated** CVE files since the last release.
- **Incremental Fetching:** For each item listed in the delta, the script performs a targeted `requests.get()` call to fetch the individual, full JSON file from GitHub using the provided `githubLink`.
- **Data Pipeline:** The fetched raw JSON passes through two key stages:
 - **Fixing:** The `fix_multi_json` function handles non-standard, concatenated JSON structures that may sometimes be present in the raw GitHub files, ensuring valid data input.
 - **Extraction/Transformation:** The `extract_important_fields` function acts as an **Extraction, Transformation, and Load (ETL) step**, paring down the verbose CVE 5.0 format to a smaller, application-specific structure.
- **Deduplication/Persistence:**
 - The `load_json_list` function acts as the **Load** step, retrieving the previously processed data.
 - The code uses `existing_ids` sets (for both new and updated lists) to prevent adding the same CVE identifier multiple times, ensuring **idempotence** for each run.
 - The `save_json_list` function overwrites the local JSON files (`new_cves.json` and `updated_cves.json`) with the complete, updated list, maintaining a valid JSON array structure.

System Architecture



Detailed Architecture Flow

1. **Trigger** - A scheduled job (cron or task scheduler) runs the script daily.
2. **Data Fetching** - The script makes a GET request to the CVE feed (NVD or CVE.org).
3. **Raw JSON Storage** - The fetched JSON is saved as `new_cves.json`
4. **Load Previous Database** - The script loads `new_cves.json` for comparison.
5. **Comparison Logic** -
 - If a CVE ID exists today but not yesterday → New CVE
 - If CVE exists but fields changed → Updated CVE
6. **Field Change Detection**
The system checks changes in:
 - CVSS score
 - Severity
 - Description
 - CWE list
 - Affected products
 - References
 - Exploit status

7. Extraction of CVSS & Severity

For updated CVEs, the system extracts:

- `cvssMetricV31.baseScore`
- `cvssMetricV31.baseSeverity`
- If missing, checks CVSSv2 fallback.

8. Email Content Creation

Emails contain sections for:

- New CVEs
- Updated CVEs
- Severity level
- Scores
- CWE mappings
- CISA links

9. Conditional Email Sending

Email will be sent only if:

New CVEs

- ✗ None
- ✓ Some
- ✗ None
- ✓ Some

Updated CVEs

- ✗ None
- ✗ None
- ✓ Some
- ✓ Some

Send Email?

- ✗ No email
- ✓ Send email
- ✓ Send email
- ✓ Send email

Code samples :

email :

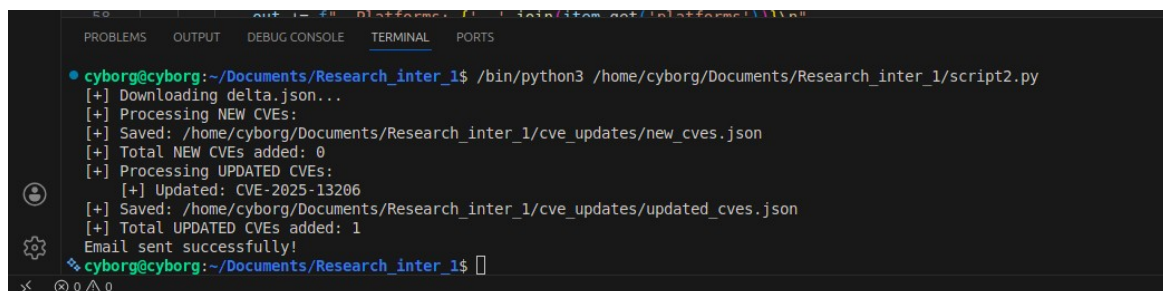
```
def send_email(sender_email, sender_password, recipient_email, subject, body):
    msg = MIMEMultipart()
    msg['From'] = sender_email
    msg['To'] = recipient_email
    msg['Subject'] = subject
    msg.attach(MIMEText(body, 'plain'))
    try:
        server = smtplib.SMTP('smtp.gmail.com', 587)
        server.starttls()
        server.login(sender_email, sender_password)
        server.send_message(msg)
        print("Email sent successfully!")
    except Exception as e:
        print(f"Failed to send email: {e}")
    finally:
        server.quit()
```

github scraping:

```
DELTA_URL = "https://raw.githubusercontent.com/CVEProject/cvelistV5/main/cves/
delta.json"
```

```
delta = requests.get(DELTA_URL).json()
```

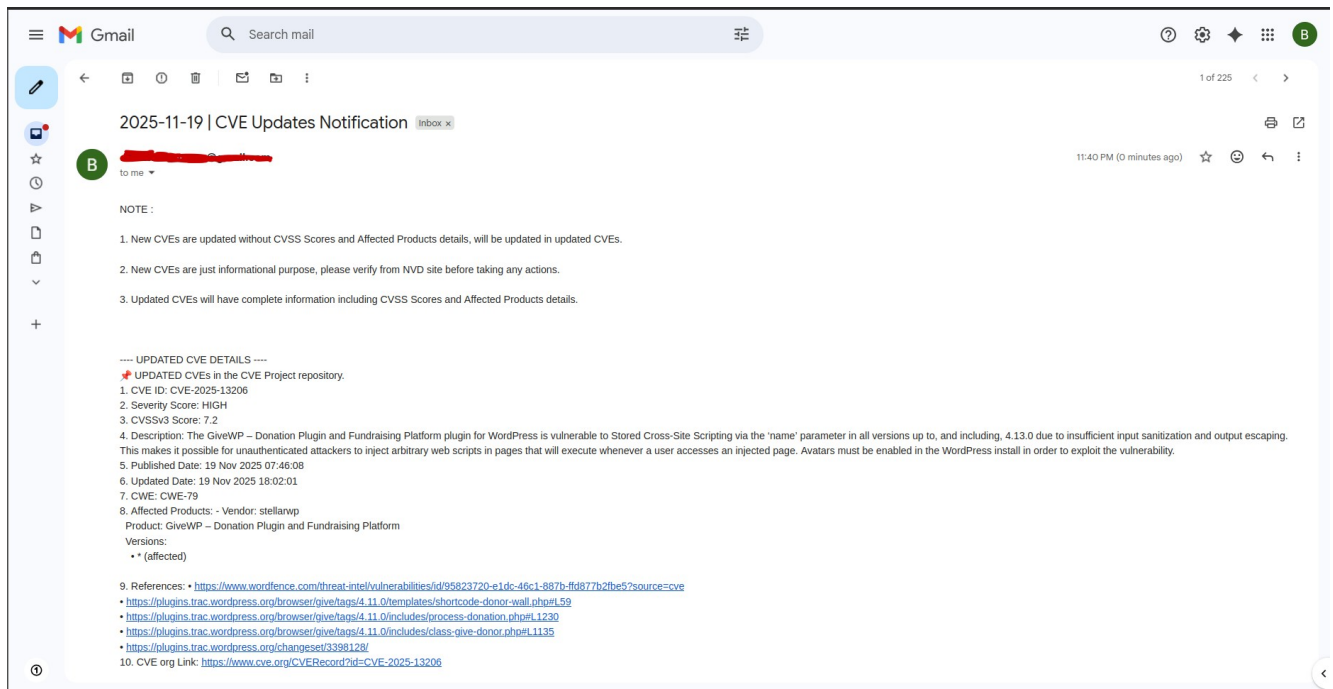
```
print(delta) # displays the new , updated cves
```



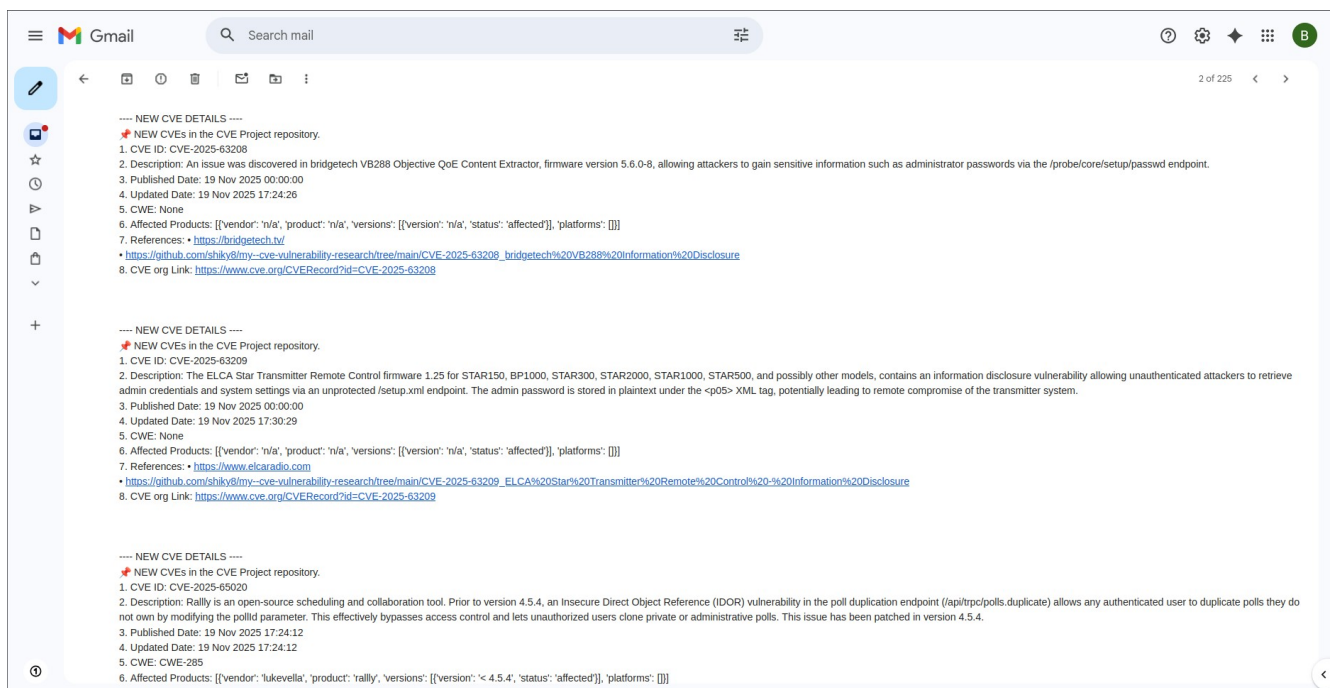
```
cyborg@cyborg:~/Documents/Research_inter_1$ /bin/python3 /home/cyborg/Documents/Research_inter_1/script2.py
[+] Downloading delta.json...
[+] Processing NEW CVEs:
[+] Saved: /home/cyborg/Documents/Research_inter_1/cve_updates/new_cves.json
[+] Total NEW CVEs added: 0
[+] Processing UPDATED CVEs:
[+] Updated: CVE-2025-13206
[+] Saved: /home/cyborg/Documents/Research_inter_1/cve_updates/updated_cves.json
[+] Total UPDATED CVEs added: 1
Email sent successfully!
cyborg@cyborg:~/Documents/Research_inter_1$
```

Sample outputs :

1. updated_cve



2. new_cves



Optional AI Integration (beginner level takes time to integrate)

Possible AI-enhanced features:

Summarization

Use GPT to convert lengthy CVE descriptions into short summaries.

Prioritization

AI can rank vulnerabilities based on:

- Exploit availability
- Affected vendor
- Product popularity
- Attack vector

RAG-based Search

Store CVE data in vector DB → allow queries like:

- “Show all RCE vulnerabilities from the last 7 days.”
- “List high severity CVEs affecting Apache.”

Threat Context Generation

AI can explain:

- Attack flow
- Real-world exploit impact
- MITRE ATT&CK mapping

This makes the tool a mini threat-intelligence assistant.